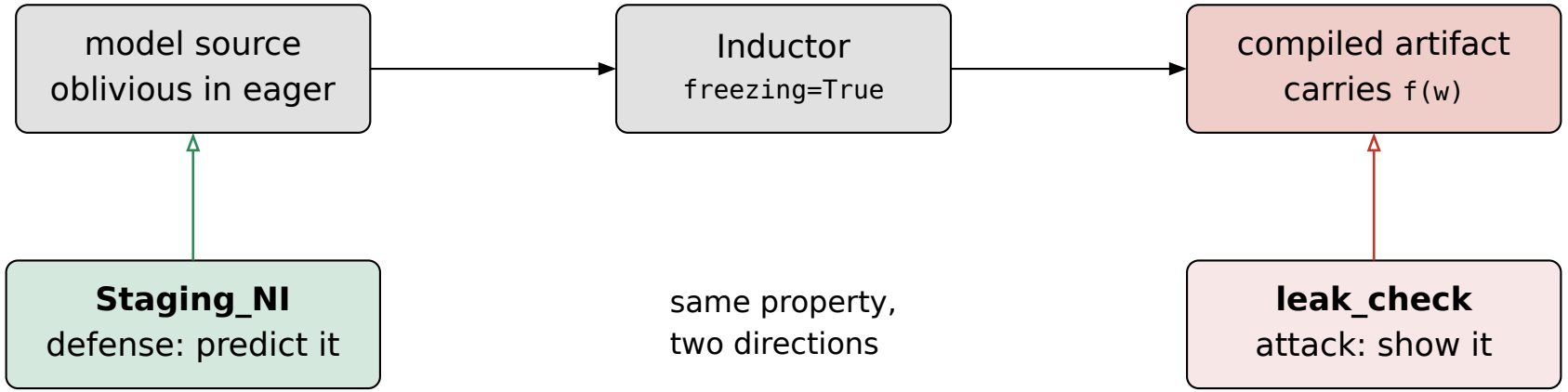
Freezing turns a weight into code

`leak_check` attacks, `Staging_NI` defends — on the GPU path

Confidential compiling stream — SPS Fellowship

Plan, not a result · `docs/research/gpu/narrative.agents.md`

The pair

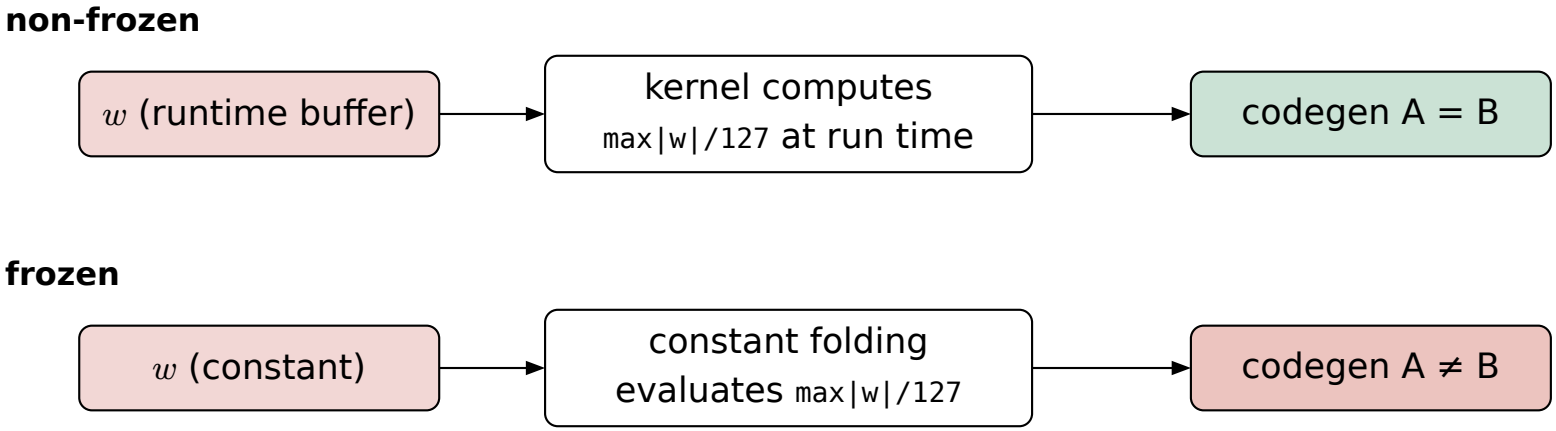


Property: **the generated artifact must not depend on protected weights**, except through explicit declassification.

Attack: `leak_check`

Measured on CPU

measured

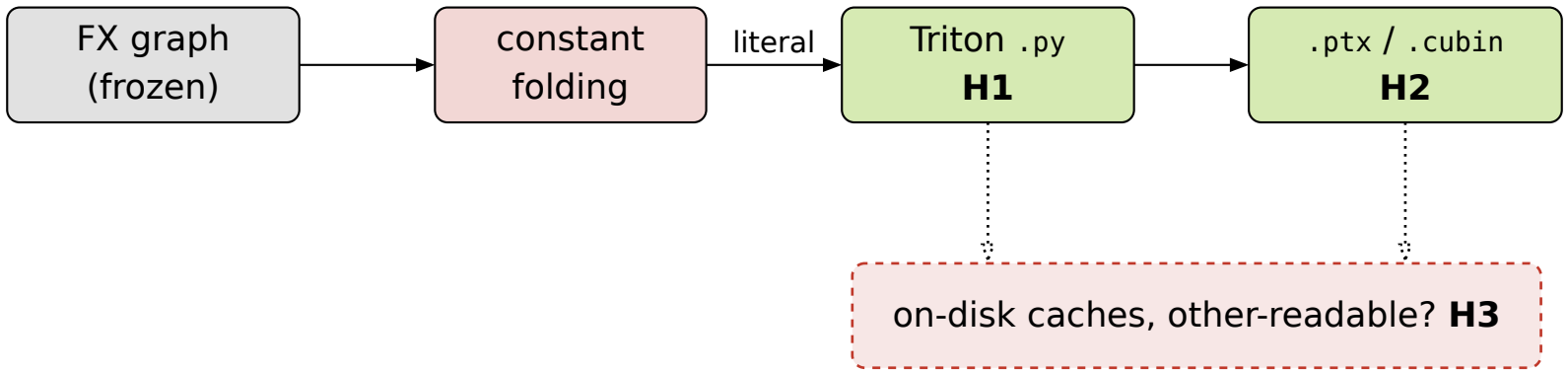



```
-   auto tmp1 = static_cast<float>(0.007874015718698502); // A
+   auto tmp1 = static_cast<float>(0.787401556968689);    // B
```

A: max|w| = 1, B: max|w| = 100. `quant_scale`, `wmax`, `wnorm` fire; `wsum` inconclusive. torch 2.13.0+cu130, Inductor CPU. `probe_freezing.py`

The GPU question

hypothesized

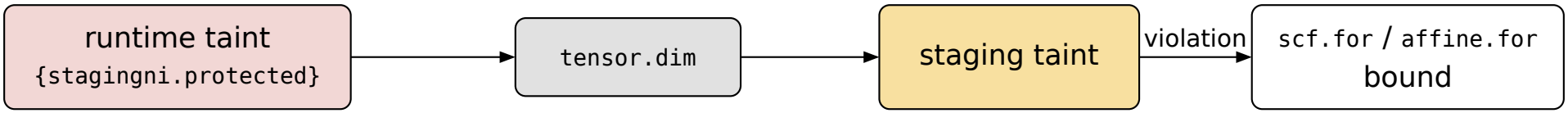


- Folding runs on the FX graph, **before** the C++ / Triton split → expect H1.
- Port: `probe_freezing.py` with `device="cuda"`, same three guards (caches off, same-class A = A′, non-frozen A = B).

Defense: Staging_NI

Staging_NI today

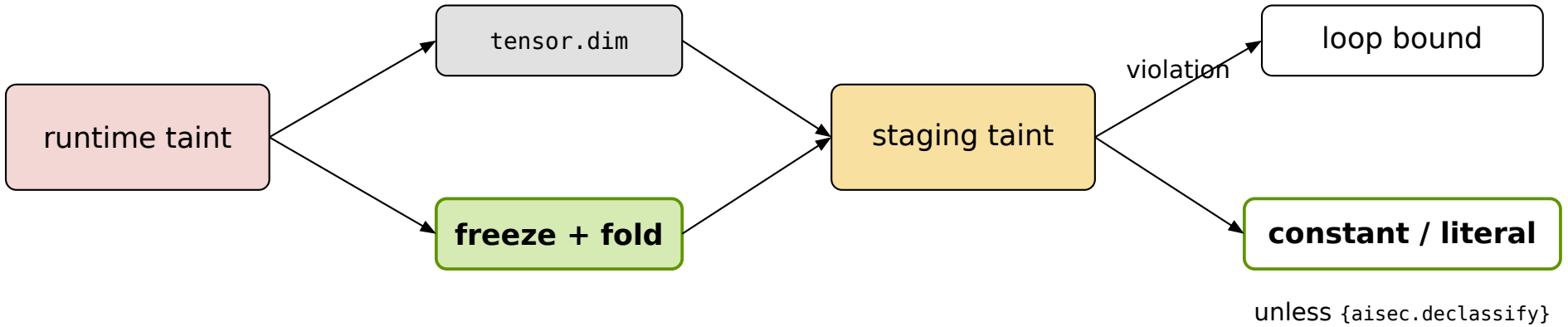
measured



- Protected **runtime** data must not decide **compile-time** structure.
- Propagation is sound over-approximation: any tainted operand taints every result; memory, regions and block arguments included; iterated to a fixpoint.
- Freezing is exactly this leak class — but `Staging_NI` has no rule for it yet.

The new rule: freezing is a conversion

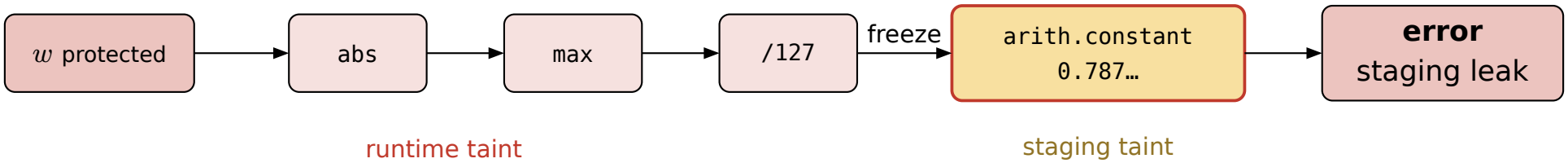
hypothesized



A foldable op with a runtime-tainted operand yields **staging** taint; a staging-tainted value materialized as a constant is a violation. Grey = existing, green = new.

Worked example: quantization scale

hypothesized



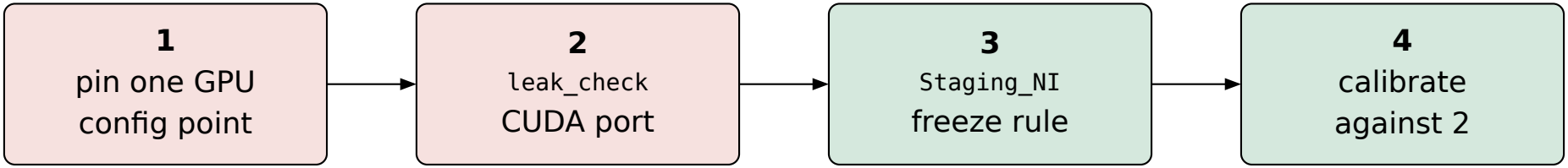
- Mark the weight `{stagingni.protected}`; the pass flags the fold **before** any code is generated.
- Repair: keep this subgraph a runtime computation, or tag the constant `{aisec.declassify}` if releasing `max|w|` is acceptable.

Calibration: the defense must agree with the attack

case	`leak_check` (dynamic)	`Staging_NI` (static)
`quant_scale`, `wmax`, `wnorm` frozen	leak measured	must fire
same surfaces, non-frozen	A = B measured	must stay silent
frozen + `{aisec.declassify}`	A ≠ B (expected)	must stay silent
`wsum` frozen	inconclusive measured	fires (over-approx.)

The static check is sound and all-inputs; the dynamic one is precise at one config point. `wsum` is where they are allowed to disagree: the leak is real in principle, the bait just can’t hold the float sum fixed.

Plan



- Steps 2 and 3 are independent and can run in parallel; step 4 needs both.
- Rule lands first at FX level (the real Inductor path), then in `Staging_NI` on MLIR output.
- Hardware: one cloud GPU. Nothing else new.

`leak_check` shows the weight in the code.

`Staging_NI` says so before compiling.